hashlock.

Security Audit

Haven1 (Layer 1)

January 2025 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

Executive Summary	4
Project Context	4
Audit scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	12
Audit Resources	12
Dependencies	12
Severity Definitions	13
Status Definitions	14
Audit Findings	15
Centralisation	18
Conclusion	19
Our Methodology	20
Disclaimers	22
About Hashlock	23

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

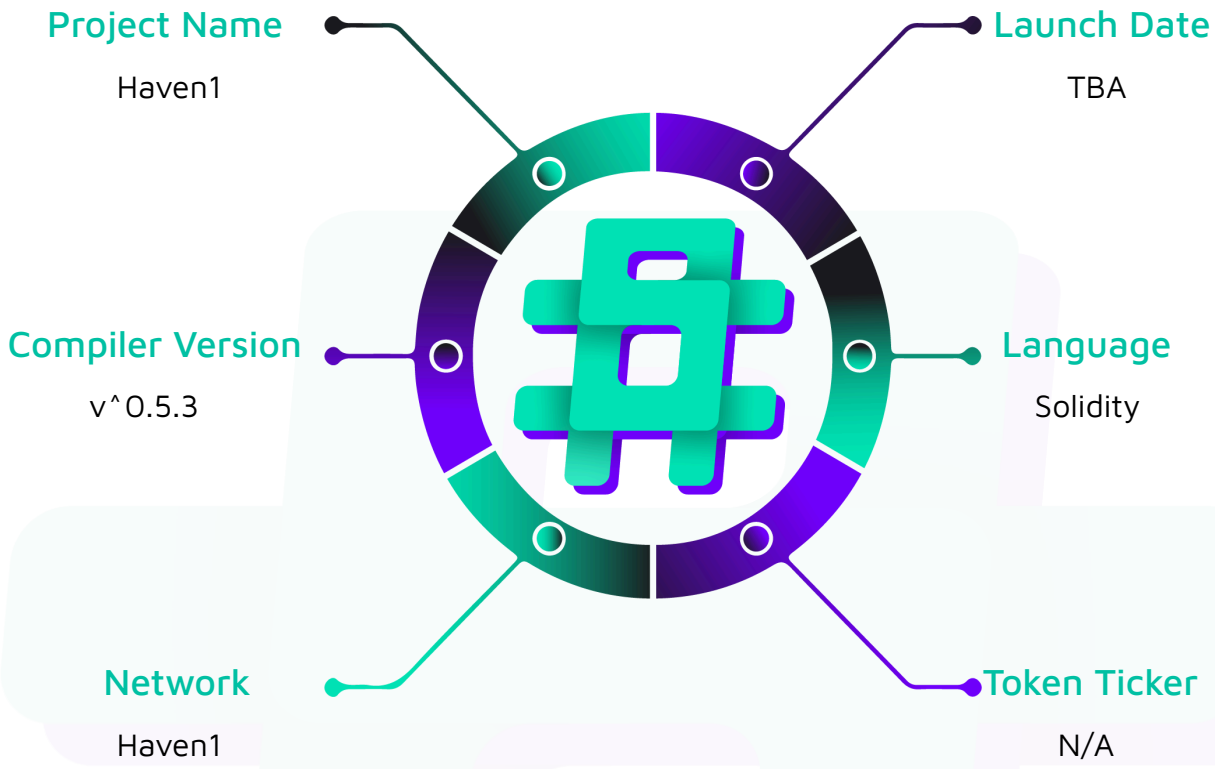
The Haven1 team partnered with Hashlock to conduct a security audit of their Go Quorum smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

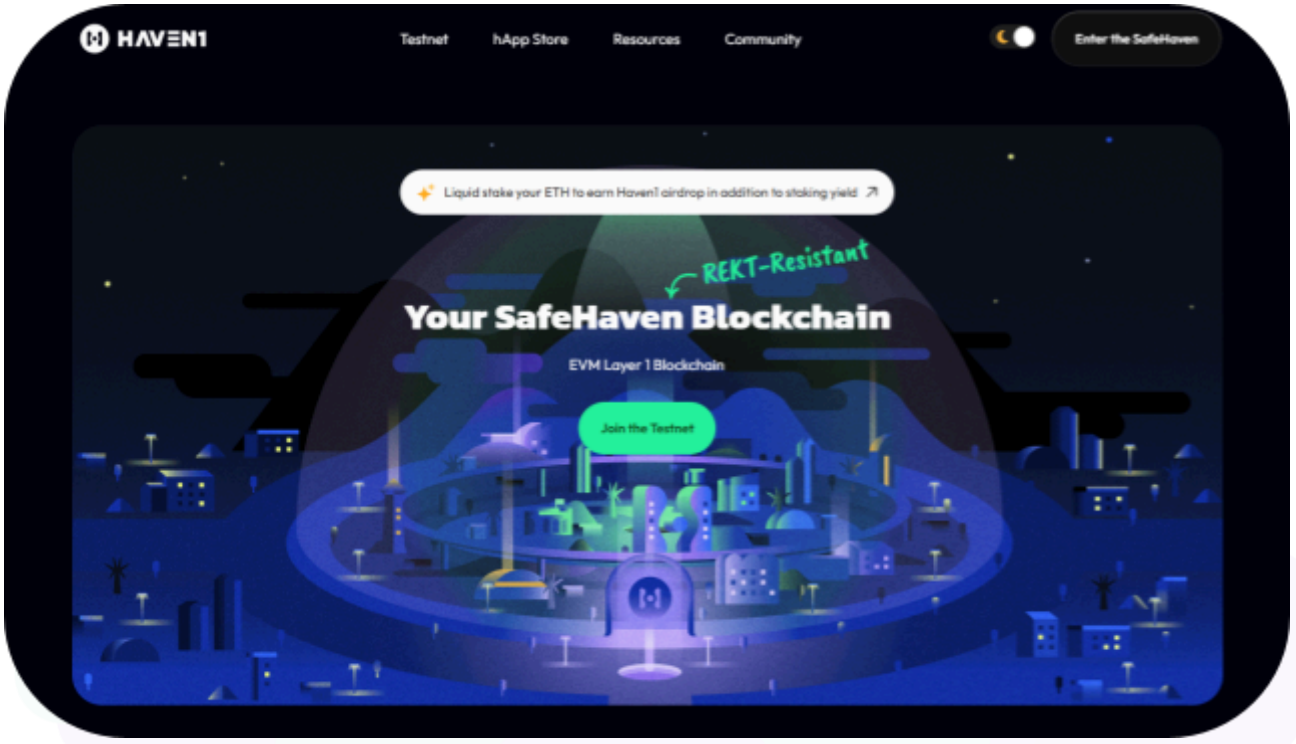
Haven1 is a REKT-resistant, EVM-compatible Layer 1 blockchain designed to safeguard users from on-chain hacks, scams, and rug pulls. By integrating advanced security measures such as the Haven1 Passport for verified access, a SafeHaven Council of reputable global validators, and 24/7 AI-powered threat monitoring, Haven1 ensures a secure environment for decentralized applications (hApps). The platform also offers innovative features like the 2FA Wallet Shield, providing on-chain two-factor authentication for enhanced asset protection. Developers benefit from EVM compatibility, comprehensive resources, and grant opportunities to build and deploy secure hApps. With a focus on deep, clean liquidity sourced from hundreds of vetted providers, Haven1 delivers a fortified SafeHaven for all participants in the Web3 ecosystem.

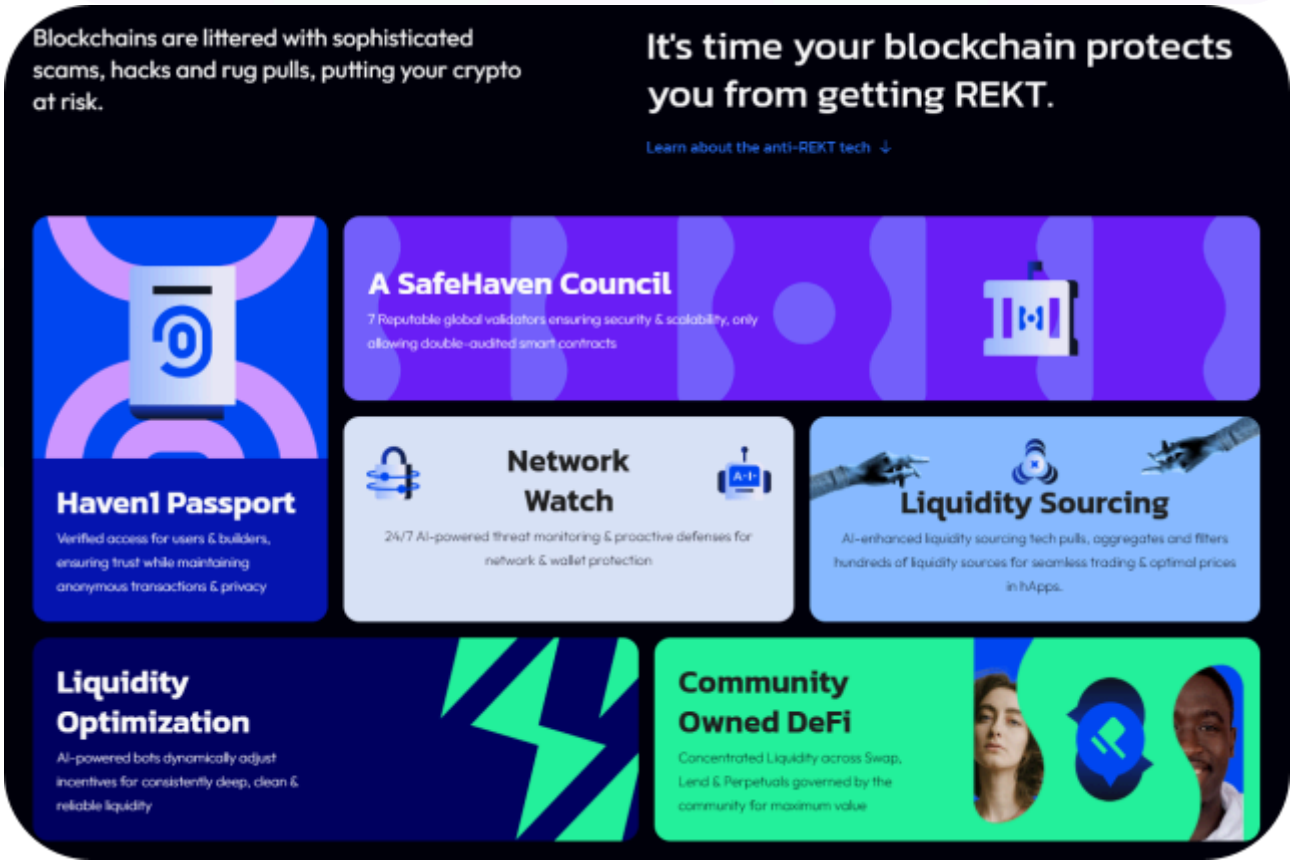
Project Name: Haven1
Project Type: Layer 1
Compiler Version: ^0.5.3
Website: https://haven1.org
Logo:

Visualised Context:



Project Visuals:





Audit scope

We at Hashlock audited the solidity code within the Haven1 project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Haven1 Network Smart Contracts
Platform	Haven1 / Solidity
Audit Date	January, 2025
Contract 1	AccountManager.sol
Contract 1 MD5 Hash	70419d4f4ec5cc16faf29d0ebfe4d9d6
Contract 2	NodeManager.sol
Contract 2 MD5 Hash	bc32207d4684d0a0ad8716134409841a
Contract 3	OrgManager.sol
Contract 3 MD5 Hash	60ca4bf342f1f6cf372d1461d59a4507
Contract 4	PermissionsImplementation.sol
Contract 4 MD5 Hash	e59d075af3869fe718b98c39f4dfca8f
Contract 5	PermissionsInterface.sol
Contract 5 MD5 Hash	3d5dbe7db65acb05d6282d9d8060d0ee
Contract 6	PermissionsUpgradable.sol
Contract 6 MD5 Hash	b0af2ca1e3128e7dd0693c8626105de6
Contract 7	RoleManager.sol
Contract 7 MD5 Hash	936a8961e3cb0b0b687c3ce2e1cb08e2
Contract 8	VoterManager.sol
Contract 8 MD5 Hash	833e22362faf5ca7ff1409957437614a

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. We initially identified some vulnerabilities that have since been addressed.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The general security overview is presented in the Standardised Checks section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved or acknowledged.

Hashlock found:

4 Low severity vulnerabilities

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
AccountManager.sol - A smart contract that manages user permissions and access control in a blockchain system through account statuses and role assignments (network/org admins) - Works with a permissions system to ensure only authorized contracts can modify account states and roles	**Contract achieves this functionality.**
NodeManager.sol - A smart contract that manages network nodes in a blockchain system, handling node statuses (active, deactivated, blacklisted) and tracking node details (enode ID, IP, ports, organization) - Controls node lifecycle operations through functions for adding, approving, and updating node status, ensuring only authorized contracts can modify node states	**Contract achieves this functionality.**
OrgManager.sol - A smart contract that manages organizational structures in a blockchain system, handling organization statuses (proposed, approved, suspended) and sub-organization hierarchies with defined depth/breadth limits - Provides functions for creating, approving, and managing organizations, their statuses, and relationships to parent/child organizations	**Contract achieves this functionality.**
PermissionsImplementation.sol	**Contract achieves this**

- A comprehensive permissions implementation contract that connects and orchestrates all the other contracts (AccountManager, RoleManager, VoterManager, NodeManager, OrgManager) to manage network access and permissions - Handles core network operations including initializing the network, managing organizations and roles, controlling node/account access, and processing admin votes - ensuring only authorized actions can be performed through a robust permission system	**functionality.**
PermissionsInterface.sol - An interface contract that acts as a gateway between external calls and the PermissionsImplementation contract, forwarding calls to the implementation while maintaining controlled access - Provides external functions for all permission management operations (org/node/role/account) but contains no core logic itself, serving purely as a standardized interface layer	**Contract achieves this functionality.**
PermissionsUpgradable.sol - A contract that manages upgradability of the permissions system by storing and controlling access to the current PermissionsImplementation and PermissionsInterface contract addresses - Only allows a designated guardian account to modify the implementation contract address and transfer guardianship, handling policy migration between old and new implementations	
RoleManager.sol	**Contract achieves this**

- A contract that manages role definitions and permissions within organizations, handling role creation, removal, and access level validation (read-only, value transfer, contract deployment, etc.) - Maintains mapping of roles to organizations and their associated access rights, with functions to check role privileges and validate transaction permissions	**functionality.**
VoterManager.sol - A contract that manages voting operations and voter permissions within organizations through functions that add/remove voters and process votes for network admin approvals - Tracks voting records and ensures proper vote counting for key governance actions like org changes, role assignments and blacklist recoveries	**Contract achieves this functionality.**

Code Quality

This audit scope involves the smart contracts of the Haven1 project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Haven1 project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Low

[L-01] NodeManager.sol - Missing array bounds check in getNodeDetailsFromIndex can return invalid data

Description

In `NodeManager.sol`, the `getNodeDetailsFromIndex` function at line 195 performs unchecked access to nodeList array via `getNodeDetailsFromIndex(uint256 _nodeIndex)`

This may revert or unnecessarily return zero values when an Index could simply be nonexistent (uninitialized).

Recommendation

Handle a case when the returned value is zero, meaning it does not exist.

Status

Acknowledged

[L-02] NodeManager.sol - Missing validation checks in addAdminNode

Description

The `addAdminNode` function in `NodeManager.sol` line `234` lacks proper validation of values such as `_ip` and `_port`. Those are defined by RFC standards. Moreover the function only checks for duplicate enodeId but does not validate IP format or port numbers, and lacks checks for duplicate IP/port combinations.

Recommendation

Implement IP format validation, reasonable port range checks, and add validation for duplicate IP/port combinations.

Status

Acknowledged

[L-03] Multiple contracts - Unnecessary access control on view functions

Description

Some of the functions, which are of view type, are implementing access control mechanisms. Since all data on the blockchain is public, restricting view functions should not be considered a security measure. At the same time, restriction on calling view functions may decrease user experience and limit protocol flexibility.

```
// OrgManager.sol

function getUltimateParent(string calldata _orgId)

    external view onlyImplementation returns (string memory)



// Similar patterns in:

// - NodeManager.sol: connectionAllowed

// - VoterManager.sol: getPendingOpDetails

// - RoleManager.sol: isVoterRole, isAdminRole
```

Recommendation

Remove the `onlyImplementation` modifier from view functions as they don't modify state and don't require access control.

Status

Acknowledged

[L-04] RoleManager.sol - Suboptimal role removal in RoleManager

Description

In `RoleManager.sol`, the removeRole function only deactivates roles rather than removing them:

```
function removeRole(
    string calldata _roleId,
    string calldata _orgId
) external onlyImplementation {
    require(
        roleIndex[keccak256(abi.encode(_roleId, _orgId))] != 0,
        "role does not exist"
    );
    uint256 rIndex = _getRoleIndex(_roleId, _orgId);
    roleList[rIndex].active = false;
    emit RoleRevoked(_roleId, _orgId);
}
```

Recommendation

Consider implementing a proper removal mechanism using array swapping and pop, or document why the current approach is necessary for historical tracking.

Status

Acknowledged

Centralisation

The Haven1 project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlock's analysis, the Haven1 project seems to have a sound and well-tested code base, now that our vulnerability findings have been acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au